LABORATORY PROGRAMS

1a. Write a python program to find the best of two test average marks out of three test marks accepted from the user.

```
# Function to get valid input for a test score
def get_valid_test_score():
    while True:
        try:
            score = float(input("Enter a test score: "))
            if 0 <= score <= 100:
                return score
            else:
                print("Please enter a valid test score between 0 and 100.")
        except ValueError:
            print("Invalid input. Please enter a valid number.")


# Input three test scores for a single student
test_scores = []

for i in range(3):
    test_scores.append(get_valid_test_score())

# Sort the test scores in descending order
test_scores.sort(reverse=True)

# Calculate the average of the best two test scores
best_two_averages = sum(test_scores[:2]) / 2

# Output the result
print(f"The average of the best two test scores is: {best_two_averages:.2f}")
```

1b. Develop a Python program to check whether a given number is palindrome or not and also count the number of occurrences of each digit in the input number.

```
def is_palindrome(number):
    original_number = number
    reversed_number = 0
    while number > 0:
        digit = number % 10
        reversed_number = reversed_number * 10 + digit
        number //= 10
    return original_number == reversed_number

def count_digits(number):
    digit_count = [0] * 10
    while number > 0:
        digit = number % 10
        digit_count[digit] += 1
        number //= 10
    return digit_count

try:
    num = int(input("Enter a number: "))
    if num < 0:
        print("Please enter a non-negative number.")
    else:
        if is_palindrome(num):
            print(f"{num} is a palindrome.")
        else:
            print(f"{num} is not a palindrome.")

        digit_count = count_digits(num)

        for digit, count in enumerate(digit_count):
            if count > 0:
                print(f"Digit {digit} appears {count} times in the number.")

except ValueError:
    print("Invalid input. Please enter a valid integer.")
```

2a. Defined as a function F as Fn = Fn-1 + Fn-2. Write a Python program which accepts a value for N (where N >0) as input and pass this value to the function. Display suitable error message if the condition for input value is not followed.

```
def fib(n):
    a=0
    b=1
    if n==0:
        print("Invalid input")
    elif n==1:
        print(a)
    else:
        print(a)
        print(b)
        for i in range(2,n):
            c=a+b
            a=b
            b=c
            print(c)
num = int(input("enter the value of n"))
fib(num)
```

2b. Develop a python program to convert binary to decimal, octal to hexadecimal using functions.

```
def binary_to_decimal(binary_str):
    decimal_value = int(binary_str, 2)
    return decimal_value

def octal_to_hexadecimal(octal_str):
    decimal_value = int(octal_str, 8)
    hexadecimal_value = hex(decimal_value).upper()
    return hexadecimal_value

def main():
    try:
        choice = input("Choose a conversion (1 for binary to decimal, 2 for octal to hexadecimal): ")

        if choice == '1':
            binary_str = input("Enter a binary number: ")
            decimal_value = binary_to_decimal(binary_str)
            print(f"Decimal equivalent: {decimal_value}")
        elif choice == '2':
            octal_str = input("Enter an octal number: ")
            hexadecimal_value = octal_to_hexadecimal(octal_str)
            print(f"Hexadecimal equivalent: {hexadecimal_value}")
        else:
            print("Invalid choice. Please enter 1 or 2.")

    except ValueError:
        print("Invalid input. Please enter a valid number.")

if __name__ == "__main__":
    main()
```

3a. Write a Python program that accepts a sentence and find the number of words, digits, uppercase letters and lowercase letters.

```
def count_characters(sentence):
    word_count = len(sentence.split())
    digit_count = sum(1 for char in sentence if char.isdigit())
    upper_count = sum(1 for char in sentence if char.isupper())
    lower_count = sum(1 for char in sentence if char.islower())

    return word_count, digit_count, upper_count, lower_count

try:
    sentence = input("Enter a sentence: ")

    word_count, digit_count, upper_count, lower_count = count_characters(sentence)

    print("Number of words:", word_count)
    print("Number of digits:", digit_count)
    print("Number of uppercase letters:", upper_count)
    print("Number of lowercase letters:", lower_count)
except ValueError:
    print("Invalid input. Please enter a valid sentence.")
```

3b. Write a Python program to find the string similarity between two given strings

Sample Output:
Original string:
Python Exercises
Python Exercises
Similarity between two said strings:
1.0

Sample Output:
Original string:
Python Exercises
Python Exercise
Similarity between two said strings:
0.967741935483871

```
import difflib
def string_similarity(str1, str2):
    result =  difflib.SequenceMatcher(a=str1.lower(), b=str2.lower())
    return result.ratio()
str1 = input("Enter the string1")
str2 = input("Enter the string2")
print("Original string:")
print(str1)
print(str2)
print("Similarity between two strings is ",string_similarity(str1,str2))
```

4a. Write a Python program to Demonstrate how to Draw a Bar Plot using Matplotlib.

```
import matplotlib.pyplot as plt

# Data for the bar plot
categories = ['Science', 'Commerce', 'Arts', 'Literature']
values = [10, 25, 15, 30]

# Create a bar plot
plt.bar(categories, values,color='m')

# Add labels and a title
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Bar Plot Example')

# Display the plot
plt.show()
```

4b. Write a Python program to Demonstrate how to Draw a Scatter Plot using Matplotlib

```
import matplotlib.pyplot as plt
import numpy as np

plt.title("Sales vs Price with Profit")

price = np.asarray([2.50, 1.23, 4.02, 3.25, 5.00, 4.40])
sales_per_day = np.asarray([34, 62, 49, 22, 13, 19])
profit = np.asarray([20, 35, 40, 20, 27.5, 15])

plt.scatter(x=price, y=sales_per_day, s=profit * 20,c='r',marker = '*',edgecolor='b')

plt.xlabel("Chocolates Price",fontsize=15)
plt.ylabel("Sales Per Day",fontsize=15)

plt.xticks(price)
plt.yticks(sales_per_day)


plt.show()
```

5a. Write a Python program to Demonstrate how to Draw a Histogram Plot using Matplotlib.

```
import matplotlib.pyplot as plt
import numpy as np

# Sample data: Student grades
grades = [75, 82, 92, 88, 78, 90, 85, 95, 70, 87, 93, 79, 81, 86, 89, 94, 73, 84, 91, 77]

# Create a histogram
plt.figure(figsize=(8, 6))
plt.hist(grades, bins=[70, 75, 80, 85, 90, 95, 100], edgecolor='black', alpha=0.7, color='skyblue')

# Customize labels and title
plt.xlabel('Grades')
plt.ylabel('Frequency')
plt.title('Grade Distribution in a Class')

# Set the x-axis limits
plt.xlim(70, 100)

# Display the plot
plt.show()
```

5b. Write a Python program to Demonstrate how to Draw a Pie Chart using Matplotlib.

```
import matplotlib.pyplot as plt

state=['Assam','Manipur','Meghalaya','Mizoram','Nagaland','Tripura','Nagaland']
forest_cover=[67353,27692,17280,17321,19240,13464,8073]

ex=[0.0,0.0,0.2,0.0,0.0,0.0,0.0]

plt.pie(forest_cover,labels=state,explode=ex,
        autopct="%0.2f%%",shadow=True,
        radius=1.0,labeldistance=1,
        startangle=90,textprops={"fontsize":10},counterclock=False,
        wedgeprops={'linewidth':1})

plt.title("Forest Cover in States of India")

plt.legend(bbox_to_anchor =(1, 0, 0.5, 1))
```

6a. Write a Python program to illustrate Linear Plotting using Matplotlib.

```
import matplotlib.pyplot as plt
import numpy as np


m=5
c=5

#when y=x
x=np.arange(-4,4)
y=np.arange(-4,4)

plt.axvline(color='g')
plt.axhline(color='g')

plt.title("Linear Plotting")

#when y=x
plt.plot(x,y,color='c',marker='D',label='Y=X')

#when y=-x
plt.plot(-x,y,color='k',marker='*',label='Y=-X')

#when slope=5 and intercept=5
plt.plot(x,m*x+c,color='r',marker='o',label='Y=5X+5')

#when slope=-5 and intercept=5
plt.plot(x,-m*x+c,color='b',marker='^',label='Y=-5X+5')

plt.grid()
plt.legend()
plt.show()
```

6b. Write a Python program to illustrate liner plotting with line formatting using Matplotlib.

```
import matplotlib.pyplot as plt
import numpy as np

# Sample data
x = np.linspace(0, 10, 100)

y1 = x
y2 = x**2
y3 = np.sin(x)

# Create a linear plot with different line formatting
plt.figure(figsize=(8, 6))

# Line 1: Solid blue line with markers
plt.plot(x, y1, label='Linear', color='blue', linestyle='-', marker='o', markersize=4, linewidth=2)

# Line 2: Dashed red line
plt.plot(x, y2, label='Quadratic', color='red', linestyle='--', linewidth=2)

# Line 3: Dotted green line with triangles
plt.plot(x, y3, label='Sine', color='green', linestyle=':', marker='^', markersize=4, linewidth=2)

  # Add labels and a legend
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Linear Plot with Line Formatting')
plt.legend()

# Display the plot
plt.grid(True)
plt.show()
```

7. Write a Python program which explains uses of customizing seaborn plots with Aesthetic functions.

a. Join Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = sns.load_dataset("iris")

# Create a joint plot for sepal length and sepal width
sns.jointplot(x="sepal_length", y="sepal_width", data=iris, kind="reg")
plt.show()
```

b. Hexagon distribution.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = sns.load_dataset("iris")

# Create a hexbin distribution plot for sepal length and sepal width
sns.jointplot(x="sepal_length", y="sepal_width", data=iris, kind="hex", color="b")
plt.title("Hexagon Distribution: Sepal Length vs. Sepal Width")
plt.show()
```

c. KDE Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = sns.load_dataset("iris")

# Create a KDE plot for petal length
sns.kdeplot(iris["petal_length"], shade=True)
plt.title("KDE Plot of Petal Length")
plt.xlabel("Petal Length (cm)")
plt.ylabel("Density")
plt.show()
```

d. Heat Map

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = sns.load_dataset("iris")

# Calculate the correlation matrix
correlation_matrix = iris.corr()

# Create a heatmap of the correlation matrix
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm")
plt.title("Heatmap of Feature Correlations in the Iris Dataset")
plt.show()
```

e. Pair Plot

```
import seaborn as sns

# Load the Iris dataset
iris = sns.load_dataset("iris")

# Create a pair plot
sns.pairplot(iris, hue="species")
plt.show()
```

f. Box Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Tips dataset
tips = sns.load_dataset("tips")

# Create a box plot for total bill amounts by day
sns.boxplot(x="day", y="total_bill", data=tips)
plt.title("Box Plot: Distribution of Total Bill Amount by Day")
plt.xlabel("Day of the Week")
plt.ylabel("Total Bill Amount ($)")
plt.show()
```

g. Regression Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Tips dataset
tips = sns.load_dataset("tips")

# Create a linear regression plot
sns.lmplot(x="total_bill", y="tip", data=tips)
plt.title("Linear Regression Plot: Total Bill vs. Tip")
plt.xlabel("Total Bill Amount ($)")
plt.ylabel("Tip Amount ($)")
plt.show()
```

h. Bar Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

# Create sample data
data = sns.load_dataset("tips")

# Customize the color palette
sns.set_palette("pastel")

# Create a bar plot
sns.barplot(x="day",y="total_bill",data=data,ci=None)
plt.title("Customized Bar Plot with Color Palette")
plt.xlabel("Day of the Week")
plt.ylabel("TotalBillAmount($)")
plt.show()
```

8. Write a Python program to explain working with bokeh line graph using Annotations and Legends.

```
from bokeh.plotting import figure, show
from bokeh.models import ColumnDataSource
from bokeh.models.annotations import Title, Legend, LegendItem
from bokeh.io import output_notebook

# Sample data
x = [1, 2, 3, 4, 5]
y1 = [2, 5, 8, 6, 7]
y2 = [4, 6, 7, 5, 9]

# Create a Bokeh figure
p = figure(title="Line Graph with Annotations and Legends", x_axis_label="X-axis", y_axis_label="Y-axis")

# Add data sources
source1 = ColumnDataSource(data=dict(x=x, y=y1))
source2 = ColumnDataSource(data=dict(x=x, y=y2))

# Plot the first line
line1 = p.line('x', 'y', source=source1, line_width=2, line_color="blue", legend_label="Line 1")

# Plot the second line
line2 = p.line('x', 'y', source=source2, line_width=2, line_color="red", legend_label="Line 2")

# Add an annotation
annotation = Title(text="Annotation Example", text_font_size="14px")
p.add_layout(annotation, 'above')

# Create a legend
legend = Legend(items=[LegendItem(label="Line 1", renderers=[line1]), LegendItem(label="Line 2", renderers=[line2])])
p.add_layout(legend, 'above')

# Show the plot
output_notebook()
show(p)
```

8a) Write a Python program for plotting different types of plots using Bokeh.

```
from bokeh.models import ColumnDataSource
from bokeh.layouts import layout
import random
import numpy as np

# Generate some sample data
x = list(range(1, 11))
y1 = [random.randint(1, 10) for _ in x]
y2 = [random.randint(1, 10) for _ in x]

# Create a Bokeh figure with custom plot dimensions
p1 = figure(title="Line Plot", x_axis_label="X-axis", y_axis_label="Y-axis", width=400, height=300)
p1.line(x, y1, line_width=2, line_color="blue")

p2 = figure(title="Scatter Plot", x_axis_label="X-axis", y_axis_label="Y-axis", width=400,height=300)
p2.scatter(x, y2, size=10, color="red", marker="circle")

p3=figure(title="BarPlot",x_axis_label="X-axis",y_axis_label="Y-axis",width=400,height=300)
p3.vbar(x=x,top=y1, width=0.5, color="green")

p4=figure(title="Histogram",x_axis_label="Value",y_axis_label="Frequency",width=400,height=300)
hist, edges = np.histogram(y1, bins=5)
p4.quad(top=hist, bottom=0, left=edges[:-1], right=edges[1:], fill_color="purple", line_color="black")

# Create a layout with the plots
layout = layout([
    [p1, p2],
    [p3, p4]
])

# Output to an HTML file
output_file("bokeh_plots.html")

# Show the plots
show(layout)
```

9a. Write a Python program to draw 3D Plots using Plotly Libraries.

i) Sine Wave

```
import seaborn as sns
from bokeh.plotting import figure, show, output_file
import plotly.graph_objects as go
import numpy as np

# Create data for the 3D surface plot
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
x, y = np.meshgrid(x, y)
z = np.sin(np.sqrt(x**2 + y**2))

# Create the 3D surface plot
fig = go.Figure(data=[go.Surface(z=z, x=x, y=y)])

# Customize the layout
fig.update_layout(
     scene=dict(
          xaxis_title='X Axis',
          yaxis_title='Y Axis',
          zaxis_title='Z Axis',
     ),
     title='3D Surface Plot Example',
)

# Show the plot
fig.show()
```

ii) Scatter Plot

```
import plotly.express as px

# Load the Iris dataset
df = px.data.iris()

# Create a 3D scatter plot
fig = px.scatter_3d(df, x="petal_length", y="petal_width", z="sepal_length", color="species",
                    title="3D Scatter Plot of Iris Dataset")
fig.update_layout(scene=dict(
    xaxis_title="Petal Length",
    yaxis_title="Petal Width",
    zaxis_title="Sepal Length"
))
```

10a. Write a Python program to draw Time Series using Plotly Libraries.

```
import plotly.express as px
import pandas as pd

df = px.data.stocks()
print(df.head(5))
fig = px.line(df, x="date", y=df.columns,
              hover_data={"date": "|%B %d, %Y"},
              title='custom tick labels')
fig.update_xaxes(
    dtick="M1",
    tickformat="%b\n%Y")
fig.show()


fig = px.scatter(df, x = "date", y = df.columns)

fig.show()

fig = px.bar(df, x = "date", y = df.columns,barmode='group')

fig.show()
```

10b. Write a Python program for creating Maps using Plotly Libraries.

```
import plotly.express as px
import pandas as pd


# Import data from USGS
data = pd.read_csv('https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_month.csv')


# Drop rows with missing or invalid values in the 'mag' column
data = data.dropna(subset=['mag'])
data = data[data.mag >= 0]


# Create scatter map
fig = px.scatter_geo(data, lat='latitude', lon='longitude', color='mag',
                     hover_name='place', #size='mag',
                     title='Earthquakes Around the World')
fig.show()
```